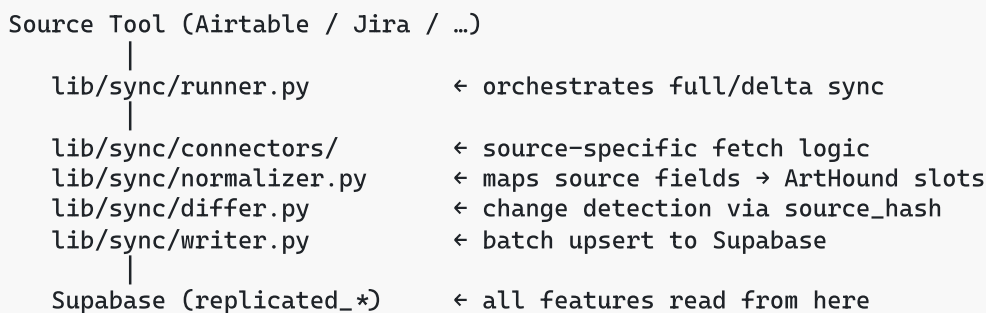


Sync Layer

Last updated: 2026-05-11

ArtHound replicates source tool data (Airtable, Jira, and future connectors) into Supabase on a continuous basis. All product features read from Supabase, never from the source tool directly. The sync layer is the engine that keeps these two worlds consistent.

Conceptual Model



A sync run is triggered in three ways, all three call the same function and are treated identically:

1. **Login trigger:** `AuthContext.jsx` fires a delta sync when a user authenticates
2. **Background polling loop:** `main.py` polls on a configurable interval (`SYNC_POLL_INTERVAL_SECONDS`)
3. **Webhook:** `POST /api/sync/webhook/{source_type}/{owner_type}/{owner_id}` receives a push notification from the source tool

Sync logic is trigger-agnostic by design. The trigger never influences what the sync does.

Sync Phases

A single sync run proceeds through six sequential phases. If any phase fails, the run is marked `error` in `sync_log` and the cursor is not advanced, so the next run retries from the last successful position.

1. Init

Loads everything the run will need from Supabase before touching the source tool:

- Studio/vendor credentials (decrypted from `source_credentials` via `lib/source_creds.py`)
- Entity definitions from `source_entity_definitions`, which source tables are Products, Assets, and Work; linking fields; filters
- Field mappings from `source_field_mappings`

- Delta cursor from `sync_cursors` (if this is a delta sync)
- Existing source hashes from `replicated_assets` (for differ)

2. Fetch

Pulls raw records from the source tool via the connector. The order is fixed: products and item types first (needed to resolve linked record names during asset normalization), then assets, then work.

For delta syncs the connector filters by modification timestamp using the stored cursor. For full syncs all records are fetched.

```
Products → fetch_entity(product_table_id, filter, since)
Item types → fetch_entity(item_type_table_id, filter, since)
Assets → fetch_entity(asset_table_id, filter, since)
Work → fetch_entity(work_table_id, filter, since)
```

After fetching, the runner builds a **reference resolver**, a lookup from source record ID → display name, populated from every fetched product, item type, and linked table. This lets asset normalization resolve linked record fields to human-readable names without additional API calls.

3. Normalize

Transforms raw source records into ArtHound's canonical shape using `lib/sync/normalizer.py`.

- Products and item types: `normalize_reference()`, extracts name + stores everything else in `meta`
- Assets: `normalize_asset()`, applies field mappings, resolves links, maps ArtHound slots, computes `source_hash`
- Work: `normalize_work()`, extracts parent asset link, derives name/status/estimate, stores remainder in `meta`

See [Field Normalization](#) below for the full slot-mapping model.

4. Diff

For delta syncs, `differ.find_changes()` compares each normalized asset's `source_hash` against the hash stored in Supabase from the previous sync. Only records whose hash changed are passed to the writer. Unchanged records are counted but skipped.

Full syncs bypass the differ and write all records.

5. Write

`lib/sync/writer.py` batches upserts to Supabase in the fixed order:

```
replicated_products → upsert_products()
replicated_item_types → upsert_item_types()
replicated_assets → upsert_assets() ← includes canonical ID minting
replicated_work → upsert_work()
```

Products and item types must exist before assets are written (FK references). Assets must exist before work (the asset canonical ID is a FK on work).

Each table is upserted in batches of 200 rows using PostgREST `resolution=merge-duplicates` . The writer passes the canonical ID map into `upsert_assets()` so canonical asset IDs are stamped on every replicated row.

Partial sync debt: Products, item types, assets, and work are written in sequence with no rollback. The cursor is not advanced on failure, so the next sync retries. But already-written phases from a failed run remain, a failure mid-write leaves the DB in a partially updated state until the next successful full sync.

6. Cleanup (full sync only)

After a full sync, `delete_orphaned_records()` identifies source record IDs that were present in Supabase but absent from the fetch results, these have been deleted from the source tool. For assets and work, orphan deletion only runs on full syncs (delta fetches are incomplete by definition). Products and item types are checked on every sync.

Deleted assets are hard-deleted from `replicated_assets` . Generated work items linked to deleted assets are soft-deleted (`deleted_at = now()`).

Field Normalization

Slot mapping

The normalizer maps source field values to ArtHound's named asset slots. Each slot has a list of **aliases**, field name patterns that are recognized as belonging to that slot:

Slot	Examples of matched names
<code>name</code>	"Asset Name", "Title", "Asset"
<code>dev_name</code>	"Dev Name", "Technical Name", "Code Name"
<code>item_type</code>	"Type", "Asset Type", "Category"
<code>priority</code>	"Priority", "Urgency"
<code>product</code>	"Product", "Project", "Game", "Show"
<code>project_date</code>	"Due Date", "Deadline", "Target Date"
<code>status</code>	"Status", "State", "Stage"
<code>asset_number</code>	"Asset #", "Asset Number", "ID"
<code>team</code>	"Team", "Department", "Studio"

Slot matching is case-insensitive. Only the first match wins. Everything that doesn't match a slot goes into `meta` (the JSONB payload).

Field mappings are persisted in `source_field_mappings` after the first sync and are editable by studios via the field mapping UI. The normalizer always resolves slots through `source_field_mappings`, it never assumes a column name from the source tool.

Meta bucket classification

Every field (whether slot-mapped or stored in meta) is classified into a **bucket** by `classify_field()`:

Bucket	Examples
<code>source_native</code>	ID fields, creation timestamps, Jira-internal fields
<code>production</code>	Status, approval, QC, priority
<code>scheduling</code>	Due dates, milestones, target dates
<code>technical</code>	Format, resolution, FPS, poly count
<code>business</code>	Contract, billing, IP
<code>custom</code>	Everything else

Classification is used by the UI to group fields in the Details tab and control progressive disclosure. Studios can override individual field buckets via `field_bucket_override_log`.

[IGNORE] fields

Any source field whose name begins with `[IGNORE]` (case-insensitive) is automatically suppressed at classification time: `ingest_suppressed=True`, `display_tier="hidden"`, excluded from `meta`. Studios use this prefix to mark internal plumbing columns (link-back columns, formula sources) that should never appear in ArtHound.

Source hash

After normalization, a SHA-256 hash of the normalized field values is stored as `source_hash`. The differ uses this hash to detect changes without comparing every field individually. A change to any field (slot or meta) produces a different hash and triggers a write.

Source Connectors

All connectors implement the `BaseConnector` interface defined in `lib/sync/connector.py`. The runner selects the correct connector based on `source_type` from the entity definition.

Airtable (`lib/sync/connectors/airtable.py`)

Uses the Airtable v0 Records API. Auth: personal access token (PAT) stored encrypted in `source_credentials`.

Fetch: `_select_all(table_name, since)` paginates via Airtable's `offset` cursor. Delta syncs add a `filterByFormula: IS_AFTER(LAST_MODIFIED_TIME(), "{since}")` clause. The `fetch_entity()` method combines studio-configured filters with the delta cursor using `AND( ... )`.

Schema: `fetch_base_schema()` returns all tables with full field definitions (id, name, type, category, options). Used by the init wizard and field mapping UI.

Write-back: `create_record(table_id, fields, qualifier_defaults)` creates records in the vendor's Airtable base. Retries on 429 (rate limit) and 5xx with exponential backoff. `qualifier_defaults` are merged into every created record to ensure PAW qualifier fields are always populated.

Attachment URLs: Airtable pre-signed S3 URLs expire in ~2 hours. The attachment pipeline re-fetches from source on expiry.

Jira (`lib/sync/connectors/jira.py`)

Supports both Jira Cloud (OAuth 2.0, REST API v3) and Jira Data Center (basic auth, REST API v2).

- **Cloud:** base URL `https://api.atlassian.com/ex/jira/{cloud_id}/rest/api/3`; cursor-based pagination (`nextPageToken`)
- **Data Center:** base URL `{instance_url}/rest/api/2`; offset pagination (`startAt` + `total`)

Fetch: `_search(jql, since)` runs JQL queries. Delta syncs append `updated ≥ "{since}"` to the JQL. Studio-configured filters from `source_entity_definitions` (either a custom `jql_filter` or `project = "{table_id}"`) are composed with the delta clause.

Schema: `fetch_base_schema()` returns all projects with the global field list and available issue types. Field types are normalized from Jira's schema format to ArtHound's via `_normalize_field()`.

Write-back: `create_issue(fields, qualifier_defaults)` creates Jira issues. `create_issue_link(link_type, inward_key, outward_key)` links issues together. Both retry on 429/5xx.

Special fields: `_jira_key` and `_jira_self` are added to every fetched record's fields dict for downstream linkage. Jira's parent/Epic Link fields are preserved as raw dicts if not explicitly mapped.

Canonical IDs

Every asset in ArtHound has a canonical UUID in `canonical_assets`. This is the stable, cross-tool identity that links all replicated data, dispatches, reviews, and work items to a single asset record.

`lib/canonical.py` provides `get_or_create_canonical_ids(source_record_ids, studio_id, source_type)`:

1. Upserts rows into `canonical_assets` on the composite key `(studio_id, source_record_id, source_type)`, safe to call repeatedly
2. Queries back the resulting UUIDs
3. Returns `{source_record_id: canonical_uuid}`

This map is passed to `upsert_assets()` during the write phase so every `replicated_assets` row is stamped with its canonical ID.

Vendor-created records (ingested from payloads) link via `payload_export_records` rather than being minted through `canonical.py`. Their canonical link back to the studio's asset is written by `_write_canonical_link()` in `routes/payload.py`.

Delta Sync and Cursors

Cursors are stored in `sync_cursors`, keyed on `(owner_type, owner_id, source_type)`. Each cursor stores `last_synced_at`, an ISO timestamp used as the delta filter passed to the connector.

After a successful sync run, the cursor is advanced to `now()` (captured at the start of the run, not the end, to avoid gaps). The cursor is only advanced after all phases succeed. On failure, the cursor stays at its previous value, so the next run retries the same window.

A **force full resync** flag (`force_full` on `sync_cursors`) causes the next run to ignore the cursor and fetch everything. This is set automatically when a partial write failure is detected mid-run.

Concurrency Control

The runner holds a per-`(owner_type, owner_id)` in-process lock. If a sync for a given owner is already in flight, new trigger attempts are logged and skipped rather than queued. This prevents overlapping syncs for the same studio/vendor from causing duplicate writes.

The lock is in-process (Python dict), not database-level. Multiple worker processes would require a distributed lock (e.g., Redis `SET NX`).

Webhook Handler

```
POST /api/sync/webhook/{source_type}/{owner_type}/{owner_id}?secret=<WEBHOOK_SECRET>
```

HMAC-verified (Airtable) or signature-verified (Jira). No JWT required, this is a public endpoint callable by the source tool.

Two behaviors:

1. **Deletion handling** (if `ENABLE_WEBHOOK_DELETIONS=true`): parses the webhook payload for deleted record IDs and immediately hard-deletes from `replicated_assets` / `replicated_work`
2. **Delta sync trigger**: fires a delta sync for the affected owner (same as the login trigger)

Deletion handling is opt-in because Airtable and Jira webhooks have different deletion payload shapes and reliability characteristics.

API Reference

All endpoints are under `/api/sync`. Protected by JWT unless noted.

Method	Path	Auth	Description
PUT	<code>/credentials</code>	studio	Save encrypted source credentials
GET	<code>/credentials</code>	studio	List configured source types (no values)
POST	<code>/run</code>	studio	Manually trigger sync; returns <code>{log_id}</code> for polling
GET	<code>/status</code>	studio	Recent sync runs + cursors for all source types
GET	<code>/status/{log_id}</code>	studio	Poll a specific run (<code>running</code> / <code>success</code> / <code>error</code>)
GET	<code>/field-mapping</code>	studio	Current mappings + slot labels + drift flag
PUT	<code>/field-mapping</code>	studio	Persist updated mappings; log bucket overrides
GET	<code>/schema-drift</code>	studio	Unresolved schema drift events
POST	<code>/webhook/{source_type}/{owner_type}/{owner_id}</code>	none (HMAC)	Webhook receiver

Background Loops (`main.py`)

Loop	Interval	Purpose
<code>_sync_poll_loop</code>	<code>SYNC_POLL_INTERVAL_SECONDS</code> (default 300s)	Delta sync for all studios with credentials
<code>_attachment_drain_loop</code>	30s, always-on	Drains <code>attachment_copy_jobs</code> queue

The sync poll loop can be disabled by setting `SYNC_POLL_INTERVAL_SECONDS=0`. The attachment drain loop is unconditional.

Schema Drift

When the source tool's schema changes (new fields, removed fields, renamed fields), `schema_drift_events` records the mismatch. The field mapping UI surfaces a drift banner when unresolved drift events exist for the studio's current mappings.

Studios resolve drift by reviewing and updating their field mappings via `PUT /api/sync/field-mapping`, which writes `field_bucket_override_log` entries for any bucket changes and clears the drift flag.

Known Gaps

No Products/item type field mapping: `source_field_mappings` only covers the Asset entity. Products and item types are normalized with a name-field heuristic (first field in the source schema) and no slot mapping beyond name. Product fields like status, owner, and deadline are never promoted to named slots, they go into `meta` only.

Partial sync write has no rollback: see Phase 5 above. The cursor protects against re-running, but a mid-run failure leaves DB state partially updated until the next full sync succeeds.

Work delta sync is incomplete: work item fetches are included in delta syncs, but work orphan deletion only runs on full syncs. Deleted work items in the source tool may persist in `replicated_work` until the next full sync.

Single-process lock: concurrency protection is in-process only. Multiple worker processes require a distributed lock.

sync_log retention, a nightly trim function keeps 100 rows per owner. Log history older than that is deleted permanently.